



Learning Python - Sheet #6

Chars & Strings

We've already worked with chars and strings without telling their name. Let's discover them in a bit more details.

1. Char

A **char** is a **character**. It can be a letter, a number or any symbol. For example 'a', '0' or '.' are chars. Some special characters are not really visible and are given special names starting with \ called **escape character**. For example '\n' which is called **line feed** and means "end of line" (or "going to the next line").

Inside the computer, the character is represented by a number called its **code**. ASCII is a very famous **character encoding** (a way to assign a code to each character). It defines how to encode 256 characters. Unicode is another character encoding which is compatible with ASCII (the first 256 characters are the same).

You can try typing the following in a new file:

```
for i in range(256):
    print("%s: %s\t" % (i, chr(i)), end='')
    if i % 16 == 15:
        print()
```

 **Pay attention:**
We're doing a lot of things in a single line (the first print), let's decompose it:

- we have **%s: %s** which means we will print 2 things
- the first thing we print is simply i, but the second thing is **chr(i)**. The function chr represents the character corresponding to a certain number.
- then we have **\t** which is an escape character for a tabulation which is a large space, useful for tabular data
- we pass **end=""** to the function print. Usually, the function print will print a new line at the end. But we can change it with the end parameter, which will tell it what to use instead (here nothing)

Every 16 value starting at 15 (that's what the **i % 16 == 15** will do), we print a new line with **print()**: we ask it to print nothing but it still uses the default end which is a new line.

This program displays the ASCII character table. You can see that the first line is a bit weird. It's cut in the middle because 10 is **\n** which goes to the next line when we print it. You can see that 0 and 9 are also messing with the alignment.

Note: IDLE is also lying to you. It displays something for characters that is not really what they're meant to look like. All characters below 32 are actually special.

There's also an **ord** method which does the reverse of **chr**. We'll use it in the next section.

2. Strings

A string is a list of characters. It can be between single quotes (') or double quotes ("). It can even be between triple-double-quotes (""") in which case they can span over multiple lines.

We've used it already every time we wanted to display a message. When we have a string, we can go over its individual characters.

```
message = input("What's your message? ")
for c in message:
    print("%s: %s" % (c, ord(c)))
```



Have fun experimenting:

What are the codes for the digits? What about the letters? Is it the same for lower case and upper case letters?



Practice to progress:

Have you heard about Caesar's cipher? It's a way to encode a message by shifting letters, often by 3 (so A becomes D, B becomes E, etc... if reaching the end of the alphabet, you loop so X becomes A, Y becomes B and Z becomes C).

Can you write a program that encode a message using Caesar's cipher?

Tip: You can create an empty string with **""** and add characters to it with **+=** (example: **result += 'a'** will add an "a" at the end of the string result)

